



CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Vanessa Christinne da Silva Siqueira

Projeto Multidisciplinar Back-End
Rede "Raíces do Nordeste"

Madrid
2026

Vanessa Christinne da Silva Siqueira

PROJETO MULTIDISCIPLINAR BACK-END
Rede "Raíces do Nordeste"

Estudo de Caso apresentado a UNINTER como
representação da capacidade de desenvolver uma
solução que comprove o domínio dos elementos
essenciais de um Back-End profissional.
Orientadora: Me. Luciane Yanase Kanashiro.
Coordenador: Me. Winston Sen Lun Fung.

Madrid
2026

RESUMO

Neste Projeto Multidisciplinar Back-End apresenta-se a integração dos conhecimentos adquiridos ao longo do curso por meio da análise e desenvolvimento de uma solução baseada em um Estudo de Caso que simula um cenário real do mercado de tecnologia. O cenário proposto contempla um contexto organizacional com regras de negócio, múltiplos usuários, integrações, requisitos de segurança, desempenho e escalabilidade, exigindo uma visão sistêmica e a aplicação dos conceitos fundamentais de Engenharia de Software. Este Estudo de Caso relaciona situações comuns enfrentadas em projetos reais, incluindo ambiguidades, prioridades concorrentes e decisões técnicas que precisam ser justificadas e documentadas.

Palavras-chave: Arquitetura em Camadas. Auditoria. JWT. LGPD. Multicanalidade.

SUMÁRIO

1	ANÁLISE E REQUISITOS	1
1.1	Requisitos Funcionais	1
1.2	Requisitos Não Funcionais	3
2	DIAGRAMAS	5
2.1	Regras de Negócio	5
2.1.1	Autenticação e Autorização	5
2.1.2	Gestão de Clientes	5
2.1.3	Gestão de Funcionários	5
2.1.4	Gestão de Lojas	5
2.1.5	Catálogo de Pratos	6
2.1.6	Gestão de Inventário	6
2.1.7	Pedidos	6
2.1.8	Auditoria	6
2.2	Diagrama de Entidade-Relacionamento	6
2.3	Diagrama de Casos de Uso	7
2.3.1	UC-01: Login no Sistema	7
2.3.2	UC-02: Cadastro de Loja	8
2.3.3	UC-03: Gestão de Funcionários	9
2.3.4	UC-04: Gestão de Inventário	10
2.3.5	UC-05: Gestão do Catálogo	12
2.3.6	UC-06: Gestão de Cliente	13
2.3.7	UC-07: Criar Pedido	15
3	ARQUITETURA	17
3.1	Visão Geral	17
3.2	Decisões Técnicas	17
3.3	Arquitetura em Camadas	18
3.3.1	Domínio	18
3.3.1.1	Estrutura Padronizada	19
3.3.1.2	Enumerações	19
3.3.2	Padrão Repository	20
3.3.3	Padrão Service	20

3.3.4	Padrões de Persistência	21
3.3.5	Fluxo de Dados	21
4	ENDPOINTS	23
4.1	Autenticação de Usuários	23
4.1.1	POST /api/v1/auth/login	23
4.1.2	POST /api/v1/auth/logout	23
4.2	Gestão de Lojas	24
4.2.1	POST /api/v1/stores	24
4.2.2	GET /api/v1/stores	24
4.2.3	GET /api/v1/stores/:store_id	25
4.2.4	PATCH /api/v1/stores/:store_id	26
4.2.5	DELETE /api/v1/stores/:store_id	27
4.3	Gestão de Funcionários	27
4.3.1	POST /api/v1/employees	27
4.3.2	GET /api/v1/employees	28
4.3.3	GET /api/v1/employees/:employee_id	29
4.3.4	PATCH /api/v1/employees/:employee_id	30
4.4	Gestão de Inventário	31
4.4.1	POST /api/v1/inventory	31
4.4.2	GET /api/v1/inventory	32
4.4.3	GET /api/v1/inventory/:item_id	33
4.4.4	PATCH /api/v1/inventory/:item_id	34
4.4.5	DELETE /api/v1/inventory/:item_id	35
4.5	Catálogo	35
4.5.1	POST /api/v1/catalogue	35
4.5.2	GET /api/v1/catalogue	36
4.5.3	GET /api/v1/catalogue/:dish_id	37
4.5.4	PATCH /api/v1/catalogue/:dish_id	37
4.5.5	DELETE /api/v1/catalogue/:dish_id	38
4.6	Gestão de Clientes	38
4.6.1	POST /api/v1/customers	38
4.6.2	GET /api/v1/customers	39
4.6.3	GET /api/v1/customers/:customer_id	40
4.6.4	PATCH /api/v1/customers/:customer_id	41
4.7	Pedidos	42
4.7.1	POST /api/v1/orders	42

4.7.2	GET /api/v1/orders	43
4.7.3	GET /api/v1/orders/me/:customer_id	44
4.7.4	GET /api/v1/orders/:order_id	45
4.8	Respostas de Erro Padrão	46
5	CONCLUSÃO	49
	Referências	51

1 ANÁLISE E REQUISITOS

1.1 Requisitos Funcionais

▪ RF1: Autenticação e Cadastro de Usuários

O sistema deve permitir o cadastro e autenticação de clientes e funcionários, com regras específicas para cada tipo de usuário. Cada perfil possui autenticação via JWT e gerenciamento de sessão, garantindo segurança e controle de acesso.

Clientes:

- Cadastro com dados pessoais (nome, email, senha, telefone e endereço);
- Email único no sistema;
- Login com email e senha;
- Logout com invalidação de sessão;
- Aceite de políticas de privacidade e termos de uso (LGPD).

Funcionários:

- Cadastro realizado apenas por administradores;
- Login com email e senha (domínio @raizesnordeste.com);
- Logout com invalidação de sessão;
- Registro de todo cadastro e acesso em log de auditoria (quem, quando, ação).

▪ RF2: Inventário de Ingredientes

O sistema deve gerenciar um inventário de ingredientes utilizados na preparação dos alimentos, permitindo controle de estoque e planejamento de compras.

- Cadastro de ingrediente (ID, nome, quantidade, quantidade mínima e medida de unidade);
- Consulta de ingredientes disponíveis;
- Atualização dos dados do ingrediente;
- Consulta da data de cadastro e última atualização de cada ingrediente;
- Consulta de ingredientes abaixo da quantidade mínima para alerta de reposição (opcional, documentado).

▪ RF3: Gestão de Cardápio

O sistema deve permitir a criação e gerenciamento de um cardápio, associando cada refeição a uma lista de ingredientes necessários para sua preparação.

- Cadastro de refeição (ID, nome, disponibilidade, preço e associação com ingredientes);
- Consulta de refeições disponíveis no cardápio;
- Atualização dos dados da refeição;
- Consulta dos ingredientes necessários para cada refeição;
- Consulta da data de cadastro e última atualização de cada refeição;
- Consulta de refeições que utilizam um ingrediente específico (opcional, documentado).

▪ RF4: Realização e Gestão de Pedidos

Pedidos registram itens solicitados, valores, status de preparo e canal de origem.

Fluxo completo: pendente → em preparo → pronto → entregue ou cancelado.

- Criação de pedido com múltiplos itens (produto, quantidade, preço unitário);
- Registro obrigatório de canal de origem (APP, TOTEM, BALCÃO, DELIVERY, WEB);
- Cálculo automático de valor total;
- Atualização de status (pendente, em preparo, pronto, entregue, cancelado);
- Cancelamento de pedido (com restrições por status);
- Consulta e filtragem de pedidos por cliente, unidade ou canal;
- Integração com serviço de pagamento (mock) no fluxo de confirmação.

▪ RF5: Programa de Fidelização

Sistema de pontos para incentivar a recompra e premiar clientes, integrado ao processo de pedidos.

- Acúmulo de pontos por pedido realizado (proporção ao valor gasto);
- Consulta de saldo de pontos do cliente;
- Resgate de pontos para desconto em pedido futuro;
- Consentimento explícito do cliente para participar do programa;
- Histórico de transações de pontos (acúmulo e resgate);
- Definição de regras de conversão (1 real = 1 ponto).

▪ RF6: Auditoria e Rastreabilidade

Registro centralizado de ações sensíveis para rastreabilidade operacional e conformidade.

- Log de operações administrativas referentes a funcionários (cadastro, acesso);

- Log de operações de clientes (cadastro, acesso, pedidos);
- Log de criação, atualização e cancelamento de pedidos;
- Identificação de autor (usuário), timestamp, ação, recurso afetado e user-agent;
- Persistência em banco de dados para consulta posterior.

1.2 Requisitos Não Funcionais

▪ RNF1: Segurança

A segurança dos dados e das operações é fundamental para proteger informações sensíveis e garantir a integridade do sistema.

- Criptografia de senhas usando bcrypt;
- Implementação de HTTP para todas as comunicações;
- Validação obrigatória de JWT em endpoints protegidos;
- Token JWT codifica a função/role do usuário para autorização;
- Controle de acesso baseado em funções (role-based access control);

▪ RNF2: Multicanalidade

O sistema deve atender múltiplos canais de interação, mantendo coerência operacional.

- Disponibilização de API REST como interface principal;
- Suporte a múltiplos canais: App, Totem, Balcão, Pickup e Web;
- Rastreabilidade de origem (canal) em todas as transações.

▪ RNF3: Auditoria e Rastreabilidade

Registro centralizado de operações críticas para conformidade e investigação.

- Logging de operações críticas de autenticação;
- Logging de todas as operações de estoque;
- Logging de operações de venda;
- Logging de ações administrativas e de funcionários;
- Registro automático de timestamp, usuário, ação, recurso afetado e código de status HTTP.

▪ RNF4: Performance e Disponibilidade

O sistema deve ser responsivo em cenários de alto uso e escalável para crescimento.

- Capacidade de lidar com horários de pico (almoço e jantar);
- Arquitetura preparada para escalabilidade horizontal (adicionar servidores);
- Tolerância a falhas em integrações externas (ex: pagamento mock).

▪ **RNF5: Lei Geral de Proteção de Dados (LGPD)**

Conformidade com a legislação brasileira de proteção de dados pessoais.

- Direito ao esquecimento: cliente pode solicitar exclusão de dados pessoais;
- Registro de consentimento do usuário ao aceitar políticas de privacidade e LGPD;
- Minimização de dados: coleta apenas do necessário (nome, email, telefone, endereço);
- Retenção limitada: dados pessoais são mantidos apenas enquanto necessário para a finalidade;
- Não exposição de dados sensíveis em respostas de API (ex: senhas, tokens internos).

2 Diagramas

2.1 Regras de Negócio

As regras de negócio definem as diretrizes e restrições que governam o funcionamento do sistema. São essenciais para garantir que o sistema atenda aos requisitos do negócio e opere de maneira eficiente e segura.

A seguir, estão descritas as principais regras de negócio para cada área do sistema.

2.1.1 Autenticação e Autorização

1. JWT Token com validade de 24 horas
2. Papéis: admin, gerente, chef, garçom, entregador
3. Restrição de acesso por loja e papel

2.1.2 Gestão de Clientes

1. Aceite obrigatório de LGPD
2. Exclusão de dados com soft delete
3. Pontos de fidelidade: 1 ponto por real gasto

2.1.3 Gestão de Funcionários

1. Apenas admin cria funcionários
2. Associação obrigatória a loja (exceto admin)
3. Exclusão de dados com soft delete

2.1.4 Gestão de Lojas

1. Apenas admin cria lojas
2. Associação de gerente obrigatório
3. Gerente adiciona funcionários à loja

2.1.5 Catálogo de Pratos

1. Apenas gerente cria e gerencia pratos
2. Preço obrigatório e positivo
3. Gestão de ingredientes via inventário

2.1.6 Gestão de Inventário

1. Apenas gerente gerencia inventário
2. Redução automática ao criar pedido
3. Associação com múltiplos pratos

2.1.7 Pedidos

1. Apenas clientes autenticados
2. Validação de existência de pratos e ingredientes
3. Cálculo de preço com taxa de 10%
4. Cálculo de pontos: $(\text{total com taxa}) \times 1$
5. Tipos: dine_in, takeaway, delivery
6. Canais: app, totem, counter, web
7. Métodos de pagamento: cash, credit_card, pix, vale_alimentacao

2.1.8 Auditoria

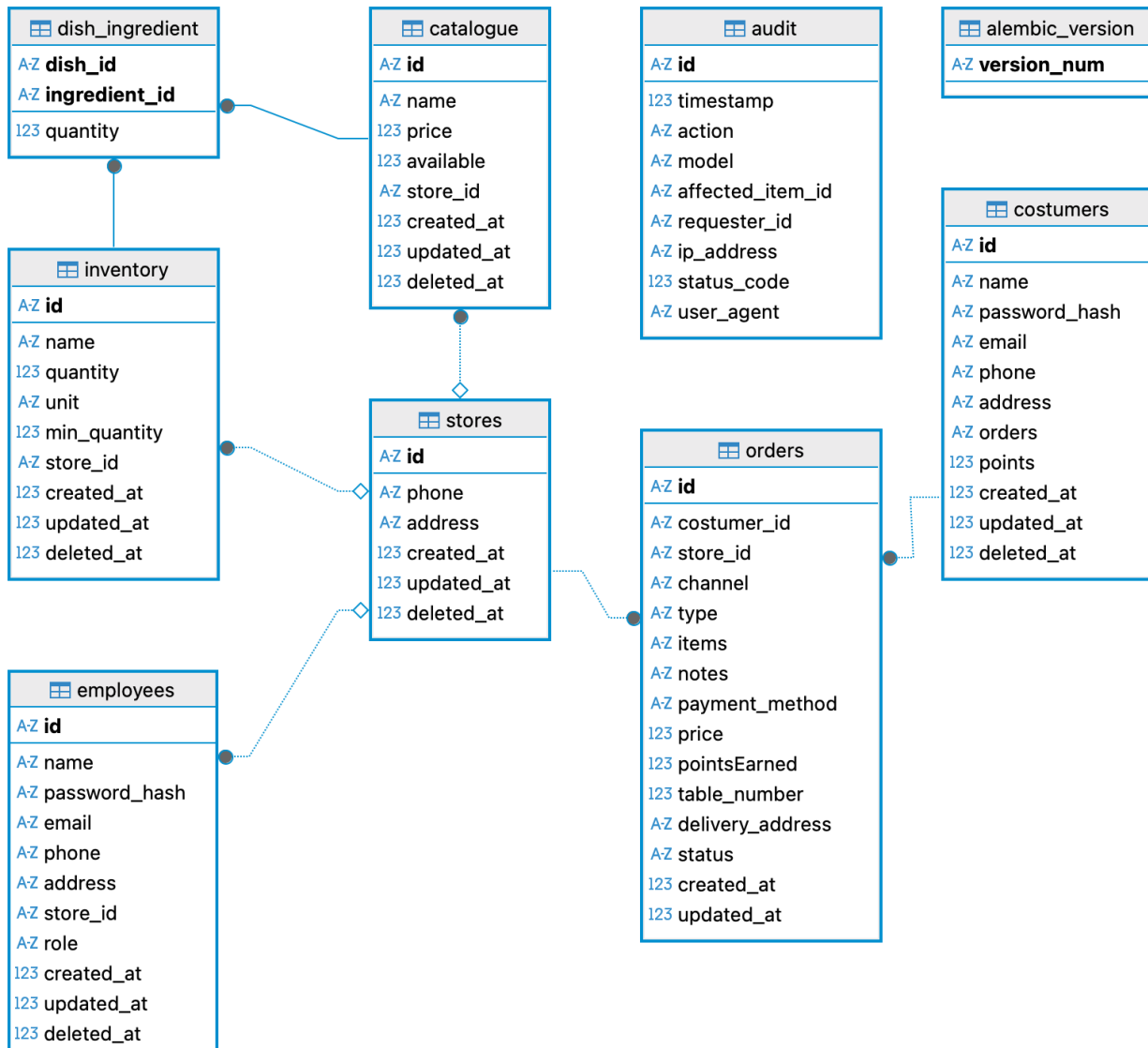
1. Todas ações registradas (create, read, update, delete, login, logout, password_change)
2. Informações: action, entity_type, entity_id, user_id, timestamp, details, ip_address
3. Não apagável (append-only)
4. Acesso restrito a admin

2.2 Diagrama de Entidade-Relacionamento

Os modelos de dados representam a estrutura das informações que serão armazenadas e manipuladas pelo sistema. Eles definem as entidades, seus atributos e os relacionamentos entre elas, garantindo a integridade e a consistência dos dados.

Na Figura 1, estão descritos as entidades principais do sistema em DER e suas respectivas descrições.

Figura 1 – DER do Raízes do Nordeste



2.3 Diagrama de Casos de Uso

2.3.1 UC-01: Login no Sistema

Descrição: Login de usuários no sistema.

Atores Principais: Administrador, Cliente e Funcionário.

Pré-condições:

1. Usuário possui conta ativa no sistema;
2. Sistema está disponível.

Pós-condições:

1. Usuário autenticado no sistema;
2. Token JWT gerado com funções apropriadas;
3. Usuário redirecionado para a página correspondente ao seu perfil.

Fluxo Principal:

1. Sistema exibe formulário solicitando email e senha;
2. Usuário preenche o formulário e submete;
3. Sistema valida as credenciais e autentica o usuário;
4. Sistema inicia sessão e redireciona para a página correspondente.

Regras de Negócio: Consulte Seção 2.1.1.

2.3.2 UC-02: Cadastro de Loja

Descrição: Cadastro de loja no sistema.

Ator Principal: Administrador.

Ator Secundário: Gerente.

Sistema Externo: API ViaCEP.

Pré-condições:

1. Administrador autenticado no sistema;
2. Loja não possui cadastro no sistema;
3. Sistema está disponível.

Pós-condições:

1. Loja registrada no sistema com dados válidos;
2. Loja pode ser gerenciada e associada a funcionários pelo Gerente;

Fluxo Principal:

1. Administrador acessa a página de cadastro de loja e preenche o formulário;

2. Sistema valida os dados e salva a loja no banco;
3. Sistema exibe mensagem de sucesso e redireciona para a lista de lojas.

Regras de Negócio: Consulte Seção 2.1.4.

2.3.3 UC-03: Gestão de Funcionários

Descrição: Administrador adiciona, visualiza, atualiza e remove funcionários do sistema.

Ator Principal: Administrador.

Atores Secundários: Gerente e Funcionário.

Sistema Externo: API ViaCEP.

Pré-condições:

1. Administrador autenticado;
2. Loja existe no sistema;
3. Sistema está disponível.

Pós-condições:

1. Funcionário criado, atualizado ou removido do sistema;
2. Permissões aplicadas conforme o papel atribuído;
3. Dados de funcionário refletem mudanças imediatamente;
4. Auditoria registra todas as operações;
5. Funcionário pode acessar sistema com email e senha.

Fluxo Principal - Criar Funcionário:

1. Administrador acessa a página de cadastro de funcionário e preenche o formulário;
2. Sistema valida os dados;
3. Funcionário é criado no banco de dados;
4. Ação é registrada na auditoria.

Fluxo Alternativo - Listar Funcionários:

1. Administrador ou Gerente acessa a lista de funcionários;
2. Gerente vê apenas funcionários de sua loja;
3. Administrador vê todos os funcionários.

Fluxo Alternativo - Visualizar Detalhes:

1. Administrador ou Gerente seleciona um funcionário;
2. Sistema retorna todas as informações.

Fluxo Alternativo - Atualizar Funcionário:

1. Administrador ou Gerente seleciona um funcionário para atualizar;
2. Sistema valida os novos dados;
3. Sistema verifica permissões:
 - a) Gerente só pode atualizar funcionários da sua loja;
 - b) Administrador pode alterar qualquer funcionário.
4. Funcionário é atualizado no banco de dados;
5. Sistema retorna os dados atualizados;
6. Ação é registrada na auditoria com o que foi alterado.

Fluxo Alternativo - Remover Funcionário¹:

1. Administrador seleciona um funcionário para remover;
2. Administrador confirma a exclusão;
3. Sistema realiza soft delete (marca deleted_at com timestamp);
4. Funcionário não pode mais fazer login;
5. Ação é registrada na auditoria.

Regras de Negócio: Consulte Seção 2.1.3.

2.3.4 UC-04: Gestão de Inventário

Descrição: Gerente de loja adiciona, visualiza, atualiza e remove itens do inventário.

Ator Principal: Gerente.

Pré-condições:

1. Gerente autenticado;
2. Loja associada ao Gerente existe;
3. Sistema está disponível.

¹ Apenas documentado, não há endpoint pois é recomendado usar soft delete para auditoria.

Pós-condições:

1. Item de inventário adicionado, atualizado ou removido;
2. Inventário reflete mudanças imediatamente;
3. Auditoria registra todas as operações;
4. Sistema alerta quando quantidade é menor que mínimo.

Fluxo Principal - Adicionar Item:

1. Gerente acessa a página de inventário e preenche o formulário para adicionar um novo item;
2. Sistema valida os dados;
3. Item é criado no inventário;
4. Sistema retorna os detalhes do item criado;
5. Ação é registrada na auditoria.

Fluxo Alternativo - Listar Itens:

1. Gerente acessa o inventário;
2. Sistema retorna lista de todos os itens do inventário da loja.

Fluxo Alternativo - Visualizar Detalhes:

1. Gerente seleciona um item específico;
2. Sistema retorna todas as informações.

Fluxo Alternativo - Atualizar Item:

1. Gerente seleciona um item para atualizar;
2. Sistema valida os novos dados;
3. Item é atualizado no banco de dados;
4. Sistema retorna os dados atualizados;
5. Ação é registrada na auditoria.

Fluxo Automático - Redução ao Criar Pedido:

1. Cliente cria um pedido com itens;
2. Sistema identifica os ingredientes necessários para cada prato;

3. Sistema retorna sucesso ou erro se não houver ingrediente suficiente.

Fluxo Alternativo - Remover Item:

1. Gerente seleciona um item para remover;
2. Gerente confirma a exclusão;
3. Sistema remove o item do inventário;
4. Sistema remove as associações com pratos (dish_ingredient);
5. Ação é registrada na auditoria.

Regras de Negócio: Consulte Seção 2.1.6.

2.3.5 UC-05: Gestão do Catálogo

Descrição: Gerente da loja adiciona, visualiza, atualiza e remove pratos do catálogo.

Ator Principal: Gerente.

Atores Secundários: Cliente.

Pré-condições:

1. Gerente autenticado;
2. Loja gerenciada pelo Gerente existe;
3. Sistema está disponível.

Pós-condições:

1. Prato adicionado, atualizado ou removido do catálogo;
2. Catálogo reflete as mudanças imediatamente;
3. Auditoria registra todas as operações;
4. Clientes veem o catálogo atualizado.

Fluxo Principal - Adicionar Prato:

1. Gerente acessa a página de catálogo e preenche o formulário para adicionar um novo prato;
2. Sistema valida os dados;
3. Sistema retorna os detalhes do prato criado;
4. Ação é registrada na auditoria.

Fluxo Alternativo - Listar Pratos:

1. Gerente ou cliente acessa o catálogo;
2. Sistema retorna lista paginada de pratos disponíveis;
3. Para cada prato: nome, preço e disponibilidade;

Fluxo Alternativo - Visualizar Detalhes do Prato:

1. Usuário seleciona um prato específico;
2. Sistema retorna todas as informações do prato;
3. Se Gerente: pode ver ingredientes associados.

Fluxo Alternativo - Atualizar Prato:

1. Gerente seleciona um prato para atualizar;
2. Sistema valida os novos dados;
3. Prato é atualizado no banco de dados;
4. Sistema retorna os dados atualizados;
5. Ação é registrada na auditoria com o que foi alterado.

Fluxo Alternativo - Remover Prato:

1. Gerente seleciona um prato para remover;
2. Sistema confirma a exclusão e processa a remoção;
3. Sistema remove as associações de ingredientes (dish_ingredient);
4. Ação é registrada na auditoria.

Regras de Negócio: Consulte Seção 2.1.5.

2.3.6 UC-06: Gestão de Cliente

Descrição: Cadastro de cliente no sistema.

Ator Principal: Cliente.

Ator Secundário: Administrador.

Sistema Externo: API ViaCEP.

Pré-condições:

1. Cliente não possui conta no sistema;

2. Sistema está disponível.

Pós-condições:

1. Cliente registrado no sistema com dados válidos;
2. Consentimento de políticas registrado;
3. Cliente pode acessar o sistema com email e senha.

Fluxo Principal - Cadastro de Cliente:

1. Cliente acessa a página de cadastro e preenche o formulário;
2. Sistema valida os dados;
3. Antes de confirmar, o cliente deve aceitar a LGPD, com consentimento registrado (data, hora, versão, IP)²;
4. Ao concluir, o sistema exibe mensagem de sucesso e redireciona para login.

Fluxo Alternativo - Listar Clientes:

1. Administrador acessa a lista de clientes;
2. Sistema retorna lista paginada de todos os clientes;

Fluxo Alternativo - Visualizar Detalhes

1. Administrador seleciona um cliente;
2. Sistema retorna todas as informações.

Fluxo Alternativo - Atualizar Cliente

1. Cliente pode atualizar seus dados pessoais;
2. Sistema salva as alterações no banco de dados.

Fluxo Alternativo - Deletar Cliente³:

1. Cliente pode solicitar exclusão de sua conta;
2. Sistema confirma a solicitação e processa a exclusão;
3. Sistema remove ou anonimiza os dados pessoais do cliente para cumprir a LGPD;
4. Sistema marca a conta como inativa (soft delete) para manter histórico e auditoria.

Regras de Negócio: Consulte Seção 2.1.2.

² Apenas documentado, não houve criação de documento de políticas, mas o sistema deve estar preparado para isso.

³ Apenas documentado, não há endpoint pois é recomendado usar soft delete para auditoria.

2.3.7 UC-07: Criar Pedido

Descrição: Cliente realiza um pedido através do sistema.

Ator Principal: Cliente

Atores Secundários: Gerente, Garçom, Entregador

Pré-condições:

1. Cliente autenticado no sistema;
2. Cliente possui conta ativa;
3. Loja está aberta e disponível;
4. Catálogo contém pratos disponíveis;

Pós-condições:

1. Pedido criado com status "pendente";
2. Preço total calculado;
3. Pontos de fidelidade atribuídos ao cliente;
4. Quantidade de ingredientes reduzida no inventário;
5. Auditoria registra a criação do pedido.

Fluxo Principal:

1. Cliente acessa o catálogo de pratos disponíveis e escolhe os itens para o pedido;
2. Cliente escolhe o método de pagamento e se deseja retirada ou entrega;
3. Sistema valida se todos os pratos existem no catálogo;
4. Sistema calcula o preço total e envia os pontos de fidelidade correspondentes;
5. Pedido é criado com status "pendente";
6. Sistema retorna os detalhes do pedido ao cliente;
7. Ação é registrada na auditoria.

Regras de Negócio: Consulte Seção 2.1.7 para regras de negócio relacionadas a pedidos e Seção 2.1.6 para redução de inventário.

3 ARQUITETURA

3.1 Visão Geral

A arquitetura do sistema “Raízes do Nordeste” segue o padrão de camadas, separando responsabilidades em níveis distintos: apresentação via rotas, lógica de negócio em serviços e persistência em repositórios.

Cada domínio segue essa estrutura de forma independente entre acesso a dados no repositório, regras de negócio no serviço e contrato HTTP nas rotas, facilitando evolução isolada de cada feature, testes unitários e refatorações sem impactos em cascata. Quanto à infraestrutura, é utilizado um banco de dados relacional com tabelas normalizadas com eficiência nas consultas.

Por fim, a estrutura de pastas do projeto reflete essa organização em camadas, com diretórios dedicados para cada domínio, além de uma seção de infraestrutura para utilitários, configuração de banco e migrações.

3.2 Decisões Técnicas

Linguagem de programação: Python

Apesar de possuir mais familiaridade técnica com Swift, Python foi escolhido por estar mais alinhado às expectativas dos avaliadores e ao padrão de projetos acadêmicos. A linguagem oferece excelente suporte a bibliotecas para autenticação, persistência e web, além de vasta comunidade e documentação, facilitando a avaliação e compreensão do código.

Framework: FastAPI

A escolha desse framework se deu por oferecer validação automática de dados via Pydantic, documentação interativa (Swagger/OpenAPI) nativa e suporte a operações assíncronas. Tais características são cruciais para construir endpoints que atendam multicanalidade (app, totem, web) e garantindo que a API seja autodocumentada conforme requisitos do projeto.

Banco de dados: SQLite e ORM

Para persistência de dados, escolheu-se SQLite por ser zero-config e adequado ao ambiente acadêmico. SQLAlchemy fornece abstração de banco de dados, facilitando migrações futuras sem afetar a lógica de negócio. SQLModel unifica modelos SQLAlchemy com schemas Pydantic, eliminando duplicação entre camadas de dados e API. Alembic complementa essa stack gerenciando versionamento de schema de forma reproduzível e auditável.

3.3 Arquitetura em Camadas

A arquitetura em camadas da aplicação segue princípios consolidados de engenharia de software, integrando padrões de design amplamente adotados na indústria. A implementação fundamenta-se nos seguintes padrões:

1. **Padrões de Design Clássicos:** Os padrões *Template Method*, *Strategy*, *Decorator* e *Dependency Injection* são classificados como padrões de design estruturais e comportamentais conforme definido por (Gamma *et al.*, 2009).
2. **Padrões Empresariais:** O padrão *Repository* e *Data Transfer Object (DTO)* são padrões de aplicação empresarial descritos por (Fowler, 2015) que facilitam a separação entre lógica de negócio e camadas de persistência.
3. **Service Layer:** O padrão de organização *Service Layer* concentra a lógica de negócio em uma camada dedicada, seguindo arquiteturas em camadas conforme descrito por (Hohpe; Woolf, 2015) e (Evans, 2014).

3.3.1 Domínio

A camada de domínio encapsula a modelagem das entidades de negócio e suas regras de validação. Esta implementação separa explicitamente as responsabilidades através do padrão DTO, Repository e Service Layer. Cada entidade é representada por um domínio específico, totalizando sete domínios principais:

Audit Registra todas as operações críticas do sistema (criação, leitura, atualização, exclusão, autenticação). Armazena metadados da auditoria, incluindo ação executada, ator responsável, recurso afetado, status HTTP, endereço IP e user agent da requisição.

Auth Gerencia autenticação e tokens JWT para acesso à API.

Catalogue Representa o cardápio de pratos oferecidos. Estabelece relacionamento muitos-para-muitos com Inventory através do modelo de ligação DishIngredient, que mapeia quais ingredientes compõem cada prato.

Costumer Modela clientes finais com autenticação, dados de contato, endereço como JSON estruturado, sistema de pontos (loyalty) e histórico de pedidos.

Employee Define funcionários com papéis de autorização (RoleEnum), dados de endereço e autenticação separada. Estrutura preparada para hierarquia de acesso baseada em roles.

Inventory Gerencia ingredientes disponíveis, incluindo nome, quantidade em estoque, unidade de medida e quantidade mínima. Permite controle de estoque e alertas para reposição.

Order Modela pedidos realizados por clientes, incluindo itens do pedido (pratos e quantidades), status, canal de origem, método de pagamento, tipo de entrega e verificação de status do pedido.

3.3.1.1 Estrutura Padronizada

Cada domínio é implementado em um arquivo único que consolida três representações complementares da mesma entidade:

- **Schema:** Classe Pydantic que valida dados de entrada para operações de criação. Define tipos, restrições de tamanho (`max_digits`, `decimal_places`), valores padrão e enumerações. Não inclui campos gerenciados pelo sistema como `id` ou `created_at`. É um DTO que transfere dados entre a API HTTP e as camadas internas.
- **UpdateSchema:** Classe Pydantic com todos os campos como `Optional`, permitindo atualizações parciais sem requerer a entidade completa. Esta separação previne validações que rejeitariam requisições legítimas com campos ausentes. Também funciona como DTO para operações de modificação.
- **Model:** Classe `SQLModel` que utiliza o decorador `table=True`, mapeando a entidade diretamente para tabelas SQL. Inclui campos de auditoria automática, UUIDs v7 como chaves primárias e relacionamentos através de `Relationship` com suporte a cardinalidade muitos-para-muitos via modelos de ligação. Não contém lógica de persistência, apenas representação de dados.

3.3.1.2 Enumerações

Para domínios com estados ou tipologias bem-definidas, enumerações Pydantic/`SQLModel` são utilizadas. Isso assegura `type-safety`, além de documentar os valores permitidos de forma explícita. Exemplos incluem:

- `RoleEnum`: Define os papéis de acesso de funcionários (`manager`, `chef`, `waiter`, `delivery`).
- `StatusEnum`: Estados de um pedido (`pending`, `preparing`, `transit`, `delivered`, `failed`, `canceled`).
- `ChannelEnum`: Canais de origem de pedidos (`app`, `totem`, `counter`, `web`).
- `OrderTypeEnum`: Modos de entrega (`dine_in`, `takeaway`, `delivery`).

- `PaymentMethodEnum`: Métodos de pagamento suportados (cash, credit_card, pix, vale_alimentacao).
- `AuditActionEnum`: Ações registradas em auditoria (create, read, update, delete, login, logout, password_change).

3.3.2 Padrão Repository

A camada Repository encapsula todas as operações de banco de dados em uma classe genérica `BaseRepository<T>`, que implementa CRUD padrão (create, read, update, delete) através de métodos type-safe. Esta separação mantém consultas SQL e gerenciamento de sessão afastados da lógica de negócio.

Cada domínio pode estender `BaseRepository` com queries específicas quando necessário. Por exemplo, `EmployeeRepository` adiciona `get_by_email()`, permitindo buscar funcionários por email sem duplicar código de base de dados. O Repository é responsável por:

- Gerenciar a sessão `SQLModel/SQLAlchemy`.
- Executar queries de forma segura.
- Controlar transações (commit/rollback).
- Retornar entidades (Models) ou None em caso de não-encontrado.

3.3.3 Padrão Service

A camada Service orquestra a transformação entre DTOs (entrada da API) e Models (persistência), aplicando lógica de negócio, validações e transformações adicionais. Cada domínio possui um Service que:

- Consome um Repository para operações de banco de dados.
- Valida regras de negócio que vão além da validação estrutural de Schemas.
- Aplica transformações de dados (ex: hashing de senhas, busca de endereço por CEP).
- Orquestra chamadas a serviços externos ou outros domínios.
- Mapeia `HTTPExceptions` para erros adequados.

A classe genérica `BaseService<ModelT, CreateSchemaT, UpdateSchemaT>` implementa operações padrão, permitindo que Services específicos adicionem métodos customizados. Por exemplo, `CostumerService.create_costumer()` não apenas cria um registro, mas também busca o endereço completo via CEP e aplica hash à senha antes de persistir.

3.3.4 Padrões de Persistência

Todas as entidades implementam soft delete através do campo `deleted_at`. Isso preserva integridade referencial e permite auditoria histórica sem perder dados. Timestamps (`created_at`, `updated_at`) são gerenciados automaticamente via `default_factory`, garantindo consistência sem requerer lógica explícita nos controladores.

Tipos de dados complexos (endereços, itens de pedidos com variações) são serializados como JSON direto no banco, aproveitando a flexibilidade do SQLite enquanto mantém queries mais simples e evita JOIN excessivos durante leitura.

Chaves primárias utilizam UUIDs v7, que combinam propriedades de UUIDs tradicionais (unicidade global) com sortabilidade temporal, favorecendo índices e paginação eficiente.

3.3.5 Fluxo de Dados

A requisição HTTP segue um fluxo claro através das camadas, garantindo separação de responsabilidades e manutenção de código:

1. **Router:** Recebe a requisição HTTP e valida contra um Schema (DTO).
2. **Service:** Recebe o Schema validado, aplica lógica de negócio, cria/atualiza um Model.
3. **Repository:** Recebe o Model e persiste no banco de dados via `SQLModel`.
4. **Resposta:** O Model é serializado novamente como JSON e retornado ao cliente.

4 ENDPOINTS

4.1 Autenticação de Usuários

4.1.1 POST /api/v1/auth/login

Finalidade: Autenticar usuários

Permissão: Público

Body: Todos os campos são obrigatórios

```
{  
  "email": "christinne@raizesnordeste.com",  
  "password": "12345678!A"  
}
```

Status Codes: 200, 401, 422

200 OK

Para fins de formatar a resposta, o token foi truncado.

```
{  
  "id": "069d510e-a565-7add-8000-65391d41016c",  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
  "token_type": "Bearer"  
}
```

4.1.2 POST /api/v1/auth/logout

Finalidade: Encerrar sessão do usuário

Permissão: JWT | Perfis: Todos os usuários autenticados

Body: Nenhum campo é necessário

Status Codes: 200, 401

200 OK

Logout simulado, num cenário real seria necessário uma blacklist de tokens.

Todos os tokens expiram em 1 hora.

```
{ "message": "Logout successful" }
```

4.2 Gestão de Lojas

4.2.1 POST /api/v1/stores

Finalidade: Criar uma nova loja

Permissão: JWT | Perfil: Admin

Body¹: Todos os campos são obrigatórios exceto complement

```
{
  "phone": "{% faker 'randomPhoneNumber' %}",
  "cep": "22775056",
  "complement": "10"
}
```

Status Codes: 201, 403, 422

201 Created

```
{
  "created_at": 1777558556,
  "phone": "(573) 327-4345 x43932",
  "deleted_at": null,
  "address": {
    "cep": "22775-056",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra da Tijuca",
    "street": "Avenida José Silva de Azevedo Neto",
    "complement": "10"
  },
  "id": "069f3641-c09e-7731-8000-b04e75d39158",
  "updated_at": 1777558556
}
```

4.2.2 GET /api/v1/stores

Finalidade: Listar todas as lojas

Permissão: JWT | Perfis: Admin

Body: Nenhum campo é necessário

¹ Para fins de teste, é usado a biblioteca Faker para gerar dados aleatórios.

Query Parameters: page e page_size

Status Codes: 200, 403, 422

200 OK

Para fins de visualização breve, a loja criada anteriormente está listada truncada.

```
{
  "data": [
    {
      "phone": "985.337.0634",
      "created_at": 1777558542,
      "deleted_at": null,
      "id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
      "updated_at": 1777912800,
      "address": {
        "cep": "22775-053",
        "state": "RJ",
        "city": "Rio de Janeiro",
        "neighborhood": "Barra Olímpica",
        "street": "Rua Amílcar de Castro",
        "complement": "255"
      }
    },
    {
      "... "
    }
  ],
  "total": 2,
  "page": 1,
  "page_size": 2,
  "total_pages": 1
}
```

4.2.3 GET /api/v1/stores/:store_id

Finalidade: Obter detalhes de uma loja específica

Permissão: JWT | Perfis: Admin

Body: Nenhum campo é necessário

Path Parameters: store_id (string, obrigatório)

Status Codes: 200, 403, 404

200 OK

```
{
  "phone": "(573) 327-4345 x43932",
  "created_at": 1777558556,
  "deleted_at": null,
  "id": "069f3641-c09e-7731-8000-b04e75d39158",
  "updated_at": 1777558556,
  "address": {
    "cep": "22775-056",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra da Tijuca",
    "street": "Avenida José Silva de Azevedo Neto",
    "complement": "10"
  }
}
```

4.2.4 PATCH /api/v1/stores/:store_id

Finalidade: Atualizar os detalhes de uma loja específica

Permissão: JWT | Perfil: Admin

Body: Todos os campos são opcionais

```
{
  "phone": "{% faker 'randomPhoneNumber' %}"
}
```

Path Parameters: store_id (string, obrigatório)

Status Codes: 200, 403, 404, 422

200 OK

Para fins de visualização breve, o address é truncado.

O phone era 985.337.0634 antes da atualização.

```
{
  "phone": "(678) 312-5573",
  "created_at": 1777558542,
  "deleted_at": null,
  "...",
  "updated_at": 1777972705,
  "id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a"
}
```

4.2.5 DELETE /api/v1/stores/:store_id

Finalidade: Excluir uma loja por ID

Permissão: JWT | Perfil: Admin

Body: Nenhum campo é necessário

Path Parameters: store_id (string, obrigatório)

Status Codes: 204, 403, 404

204 No Content

Loja excluída com sucesso. Nenhum conteúdo é retornado.

4.3 Gestão de Funcionários**4.3.1 POST /api/v1/employees**

Finalidade: Criar um novo funcionário

Permissão: JWT | Perfil: Admin

Body: Todos os campos são obrigatórios exceto complement

```
{
  "name": "{{ _.firstName }} {{ _.lastName }}",
  "password": "12345678!D",
  "email": "{{ _.firstName | lower }}.{{ _.lastName | lower }}@{{ _.companyEmail }}",
  "phone": "% faker 'randomPhoneNumber' %",
  "cep": "22775040",

```

```
"complement": "AP 21",
"store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
"role": "manager"
}
```

Status Codes: 200, 401, 403, 422

201 Created

```
{
  "phone": "1-341-646-0971 x0509",
  "name": "Larry Larson",
  "password_hash": "$5$rounds=1000$QG.hkdsv4bB5ox1t$c0Smfn/33pt7IHiMSP3...",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "created_at": 1777976616,
  "deleted_at": null,
  "id": "069f9c52-8d7b-7949-8000-56f1c07bd145",
  "email": "larry.larson@raizesnordeste.com",
  "address": {
    "cep": "22775-040",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra Olímpica",
    "street": "Avenida Embaixador Abelardo Bueno",
    "complement": "AP 21"
  },
  "role": "manager",
  "updated_at": 1777976616
}
```

4.3.2 GET /api/v1/employees

Finalidade: Listar todos os funcionários registrados

Permissão: JWT | Perfis: Admin, Manager

Body: Nenhum campo é necessário

Query Parameters: page e page_size

Status Codes: 200, 401, 403, 422

200 OK

Para fins de visualização breve, alguns funcionários estão listados truncados.

```
{
  "data": [
    {
      "name": "Christinne",
      "store_id": null,
      "created_at": 1777555453,
      "id": "069f357f-dc5e-7386-8000-543f9b640132",
      "role": "admin",
      "..."
    },
    {
      "created_at": 1777976616,
      "id": "069f9c52-8d7b-7949-8000-56f1c07bd145",
      "..."
    },
    {
      "created_at": 1778056903,
      "id": "069fafec-7f01-741e-8000-08bd7a855107",
      "..."
    }
  ],
  "total": 3,
  "page": 1,
  "page_size": 10,
  "total_pages": 1
}
```

4.3.3 GET /api/v1/employees/:employee_id

Finalidade: Obter detalhes de um funcionário específico

Permissão: JWT | Perfis: Admin, Manager

Body: Nenhum campo é necessário

Path Parameters: employee_id (string, obrigatório)

Status Codes: 200, 401, 403, 404

200 OK

```
{
  "name": "Jared Hackett",
  "password_hash": "$5$rounds=1000$QG.hkdsv4bB5ox1t$c0Smfn/33pt7IHIMSP3uU ...",
  "phone": "(305) 947-7813 x04031",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "created_at": 1778056903,
  "deleted_at": null,
  "email": "jared.hackett@raizesnordeste.com",
  "id": "069fafec-7f01-741e-8000-08bd7a855107",
  "address": {
    "cep": "22775-046",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra Olímpica",
    "street": "Rua Mário Agostinelli",
    "complement": "AP 10"
  },
  "role": "waiter",
  "updated_at": 1778056903
}
```

4.3.4 PATCH /api/v1/employees/:employee_id

Finalidade: Atualizar detalhes de um funcionário específico

Permissão: JWT | Perfil: Admin, Manager (apenas para funcionários da mesma loja)

Body: Todos os campos são opcionais

```
{
  "name": "{{ _.firstName }} {{ _.lastName }}",
  "email": "{{ _.firstName | lower }}.{{ _.lastName | lower }}@{{ _.companyEmail }}",
  "phone": "{{% faker 'randomPhoneNumber' %}}"
}
```

Path Parameters: employee_id (string, obrigatório)

Status Codes: 200, 401, 403, 404, 422

200 OK

```
{
  "phone": "317.390.8938 x305",
  "name": "Ida Kozey",
  "password_hash": "$5$rounds=1000$QG.hkdsv4bB5ox1t$c0Smfn/33pt7IHIMSP3uU...",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "created_at": 1777976616,
  "deleted_at": null,
  "id": "069f9c52-8d7b-7949-8000-56f1c07bd145",
  "email": "ida.kozey@raizesnordeste.com",
  "address": {
    "cep": "22775-040",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra Olímpica",
    "street": "Avenida Embaixador Abelardo Bueno",
    "complement": "AP 21"
  },
  "role": "manager",
  "updated_at": 1778076350
}
```

4.4 Gestão de Inventário

4.4.1 POST /api/v1/inventory

Finalidade: Adicionar um novo item ao inventário

Permissão: JWT | Perfil: Manager

Body: Todos os campos são obrigatórios

```
{
  "name": "Frango",
  "quantity": 5000,
  "unit": "g",
  "min_quantity": 1000,
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a"
}
```

Status Codes: 201, 401, 403, 422

201 Created

```
{
  "unit": "g",
  "id": "069fb14d-a64e-79b3-8000-7e9fb7ca8730",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778062554,
  "name": "Frango",
  "quantity": "5000.000",
  "min_quantity": "1000.000",
  "created_at": 1778062554,
  "deleted_at": null
}
```

4.4.2 GET /api/v1/inventory

Finalidade: Listar itens do inventário da loja associada ao usuário autenticado

Permissão: JWT | Perfil: Manager

Body: Nenhum campo é necessário

Status Codes: 200, 401, 403

200 OK

Para fins de visualização breve, o item criado anteriormente está listado truncado.

```
[
  {
    "...",
  },
  {
    "unit": "g",
    "id": "069fb465-548f-7cf8-8000-984bc84a85e6",
    "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
    "updated_at": 1778075221,
    "name": "Pão de Forma",
    "quantity": "3000.000",
    "min_quantity": "500.000",
    "created_at": 1778075221,
    "deleted_at": null
  }
]
```

4.4.3 GET /api/v1/inventory/:item_id

Finalidade: Obter detalhes de um item específico do inventário

Permissão: JWT | Perfil: Manager

Body: Nenhum campo é necessário

Path Parameters: item_id (string, obrigatório)

Status Codes: 200, 401, 403, 404

200 OK

```
{
  "unit": "g",
  "id": "069fb465-548f-7cf8-8000-984bc84a85e6",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778075221,
  "name": "Pão de Forma",
  "quantity": "3000.000",
  "min_quantity": "500.000",
  "created_at": 1778075221,
  "deleted_at": null
}
```

4.4.4 PATCH /api/v1/inventory/:item_id

Finalidade: Atualizar um item específico do inventário

Permissão: JWT | Perfil: Manager

Body: Todos os campos são opcionais

```
{
  "name": "Pão de Forma Tradicional"
}
```

Path Parameters: item_id (string, obrigatório)

Status Codes: 200, 401, 403, 404, 422

200 OK

```
{
  "unit": "g",
  "id": "069fb465-548f-7cf8-8000-984bc84a85e6",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778084191,
  "name": "Pão de Forma Tradicional",
  "quantity": "3000.000",
  "min_quantity": "500.000",
  "created_at": 1778075221,
  "deleted_at": null
}
```


4.4.5 DELETE /api/v1/inventory/:item_id

Finalidade: Excluir um item específico do inventário

Permissão: JWT | Perfil: Manager

Body: Nenhum campo é necessário

Path Parameters: item_id (string, obrigatório)

Status Codes: 204, 401, 403, 404

204 No Content

Item excluído com sucesso. Nenhum conteúdo é retornado.

4.5 Catálogo

4.5.1 POST /api/v1/catalogue

Finalidade: Adicionar um novo prato ao catálogo

Permissão: JWT | Perfil: Manager

Body: Todos os campos são obrigatórios

```
{
  "name": "Hambúrguer com Batata Frita",
  "price": 25.90,
  "available": true,
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a"
}
```

Status Codes: 201, 403, 422

201 Created

```
{
  "price": "25.90",
  "name": "Hambúrguer com Batata Frita",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778085006,
  "available": true,
  "id": "069fb6c8-e763-7d3e-8000-39308786ceea",
  "created_at": 1778085006,
  "deleted_at": null
}
```

4.5.2 GET /api/v1/catalogue

Finalidade: Listar todos os produtos do catálogo

Permissão: Público

Body: Nenhum campo é necessário

Query Parameters: page e page_size

Status Codes: 200, 422

200 OK

```
{
  "data": [
    {
      "price": "25.90",
      "id": "069fb6c8-e763-7d3e-8000-39308786ceea",
      "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
      "updated_at": 1778085006,
      "available": true,
      "name": "Hambúrguer com Batata Frita",
      "created_at": 1778085006,
      "deleted_at": null
    },
    {
      "price": "10.00",
      "id": "069fb715-30c8-7a37-8000-0f19c23558b4",
      "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
      "updated_at": 1778086227,
      "available": true,
      "name": "Brigadeiro (6un)",
      "created_at": 1778086227,
      "deleted_at": null
    }
  ],
  "total": 2,
  "page": 1,
  "page_size": 10,
  "total_pages": 1
}
```

4.5.3 GET /api/v1/catalogue/:dish_id

Finalidade: Obter detalhes de um prato específico

Permissão: Público

Body: Nenhum campo é necessário

Path Parameters: item_id

Status Codes: 200, 404

200 OK

```
{
  "price": "10.00",
  "id": "069fb715-30c8-7a37-8000-0f19c23558b4",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778086227,
  "available": true,
  "name": "Brigadeiro (6un)",
  "created_at": 1778086227,
  "deleted_at": null
}
```

4.5.4 PATCH /api/v1/catalogue/:dish_id

Finalidade: Atualizar os detalhes de um prato específico

Permissão: JWT | Perfil: Manager

Body: Todos os campos são opcionais

```
{
  "price": 9.50
}
```

Path Parameters: item_id

Status Codes: 200, 403, 404, 422

200 OK

```
{
  "price": "9.50",
  "id": "069fb715-30c8-7a37-8000-0f19c23558b4",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "updated_at": 1778086442,
  "available": true,
  "name": "Brigadeiro (6un)",
  "created_at": 1778086227,
  "deleted_at": null
}
```

4.5.5 DELETE /api/v1/catalogue/:dish_id

Finalidade: Remover um prato do catálogo

Permissão: JWT | Perfil: Manager

Body: Nenhum campo é necessário

Path Parameters: item_id

Status Codes: 204, 403, 404

204 No Content

Prato excluído com sucesso. Nenhum conteúdo é retornado.

4.6 Gestão de Clientes

4.6.1 POST /api/v1/customers

Finalidade: Criar um novo cliente

Permissão: Público

Body: Todos os campos são obrigatórios exceto complement

```
{
  "name": "{{ _.firstName }} {{ _.lastName }}",
  "password": "12345678!C",
  "email": "{{ _.firstName }}{{ _.lastName }}@gmail.com",
  "phone": "{% faker 'randomPhoneNumber' %}",
  "cep": "22775055",
  "complement": "string",
}
```

```
"orders": [  
  null  
],  
"points": 0  
}
```

Status Codes: 201, 422

201 Created

```
{  
  "name": "Cory Balistreri",  
  "email": "CoryBalistreri@gmail.com",  
  "address": {  
    "cep": "22775-055",  
    "state": "RJ",  
    "city": "Rio de Janeiro",  
    "neighborhood": "Barra da Tijuca",  
    "street": "Avenida Luís Carlos Prestes",  
    "complement": "AP 42"  
  },  
  "points": 0,  
  "updated_at": 1778088064,  
  "id": "069fb788-0940-7431-8000-c1d2a353b3f0",  
  "phone": "737-209-4403 x53519",  
  "password_hash": "$5$rounds=1000$kHW63hGpr9eo0vm4$Ev1Y.hJ0S9ZN9QQ8abL9I...",  
  "orders": [],  
  "created_at": 1778088064,  
  "deleted_at": null  
}
```

4.6.2 GET /api/v1/customers

Finalidade: Listar todos os clientes registrados

Permissão: JWT | Perfil: Admin

Body: Nenhum campo é necessário

Status Codes: 200, 401, 403

200 OK

Para fins de visualização breve, o cliente criado anteriormente está listado truncado.

```
{
  "data": [
    {
      "...",
    },
    {
      "name": "Danyka Keebler",
      "email": "DanykaKeebler@outlook.com",
      "address": {
        "cep": "22775-055",
        "state": "RJ",
        "city": "Rio de Janeiro",
        "neighborhood": "Barra da Tijuca",
        "street": "Avenida Luís Carlos Prestes",
        "complement": "AP 10"
      },
      "points": 0,
      "updated_at": 1778088253,
      "id": "069fb793-d9b9-725b-8000-597c91f9a534",
      "phone": "1-274-705-2264 x964",
      "password_hash": "$5$rounds=1000$Ydnx7U24QRW8Z/T6$wbVjQkzc1GnLkdDp7...",
      "orders": [],
      "created_at": 1778088253,
      "deleted_at": null
    }
  ],
  "total": 2,
  "page": 1,
  "page_size": 10,
  "total_pages": 1
}
```

4.6.3 GET /api/v1/customers/:customer_id

Finalidade: Obter detalhes de um cliente específico

Permissão: JWT | Perfil: Admin

Body: Nenhum campo é necessário

Status Codes: 200, 401, 403, 404

200 OK

```
{
  "name": "Cory Balistreri",
  "email": "CoryBalistreri@gmail.com",
  "address": {
    "cep": "22775-055",
    "state": "RJ",
    "city": "Rio de Janeiro",
    "neighborhood": "Barra da Tijuca",
    "street": "Avenida Luís Carlos Prestes",
    "complement": "AP 42"
  },
  "points": 0,
  "updated_at": 1778088064,
  "id": "069fb788-0940-7431-8000-c1d2a353b3f0",
  "phone": "737-209-4403 x53519",
  "password_hash": "$5$rounds=1000$kHW63hGpr9eo0vm4$Ev1Y.hJ0S9ZN9QQ8abL9I...",
  "orders": [],
  "created_at": 1778088064,
  "deleted_at": null
}
```

4.6.4 PATCH /api/v1/customers/:customer_id

Finalidade: Atualizar informações de um cliente específico

Permissão: JWT | Perfil: Cliente

Body: Todos os campos são opcionais

```
{
  "name": "{{ _.firstName }} {{ _.lastName }}",
  "email": "{{ _.firstName }}{{ _.lastName }}@gmail.com"
}
```

Status Codes: 200, 401, 403, 404, 422

200 OK

Para fins de visualização breve, alguns campos foram truncados.

```
{
  "name": "Reed Schoen",
  "email": "ReedSchoen@gmail.com",
  "updated_at": 1778090958,
  "id": "069fb788-0940-7431-8000-c1d2a353b3f0",
  "... "
}
```

4.7 Pedidos

4.7.1 POST /api/v1/orders

Finalidade: Criar um novo pedido

Permissão: JWT | Perfil: Cliente

Body: Todos os campos são obrigatórios exceto notes, table_number e delivery_address

```
{
  "channel": "app",
  "type": "delivery",
  "items": [
    {
      "name": "Hambúrguer com Batata Frita",
      "quantity": 1
    },
    {
      "name": "Brigadeiro (6un)",
      "quantity": 1
    }
  ],
  "notes": "",
  "payment_method": "pix"
}
```

Status Codes: 201, 422

201 Created

```
{
  "channel": "app",
  "id": "069fbafd-9f9d-77fd-8000-f9429e2381b0",
  "items": [
    {
      "name": "Hambúrguer com Batata Frita",
      "quantity": 1
    },
    {
      "name": "Brigadeiro (6un)",
      "quantity": 1
    }
  ],
  "payment_method": "pix",
  "pointsEarned": 38,
  "delivery_address": null,
  "status": "pending",
  "updated_at": 1778102233,
  "type": "delivery",
  "costumer_id": "069fb788-0940-7431-8000-c1d2a353b3f0",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "notes": "",
  "price": "38.94",
  "table_number": null,
  "created_at": 1778102233
}
```

4.7.2 GET /api/v1/orders

Finalidade: Listar todos os pedidos realizados na loja administrada pelo manager autenticado

Permissão: JWT | Perfil: Manager, Waiter, Delivery

Status Codes: 200, 401, 403

200 OK

```
[
  {
    "channel": "app",
    "id": "069fbafd-9f9d-77fd-8000-f9429e2381b0",
    "items": [
      {
        "name": "Hambúrguer com Batata Frita",
        "quantity": 1
      },
      {
        "name": "Brigadeiro (6un)",
        "quantity": 1
      }
    ],
    "payment_method": "pix",
    "pointsEarned": 38,
    "delivery_address": null,
    "status": "pending",
    "updated_at": 1778102233,
    "type": "delivery",
    "costumer_id": "069fb788-0940-7431-8000-c1d2a353b3f0",
    "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
    "notes": "",
    "price": "38.94",
    "table_number": null,
    "created_at": 1778102233
  }
]
```

4.7.3 GET /api/v1/orders/me/:customer_id

Finalidade: Lista todos os pedidos realizados pelo cliente

Permissão: JWT | Perfil: Cliente

Status Codes: 200, 401, 403, 404

200 OK

```
[
  {
    "channel": "app",
    "id": "069fbafd-9f9d-77fd-8000-f9429e2381b0",
    "items": [
      {
        "name": "Hambúrguer com Batata Frita",
        "quantity": 1
      },
      {
        "name": "Brigadeiro (6un)",
        "quantity": 1
      }
    ],
    "payment_method": "pix",
    "pointsEarned": 38,
    "delivery_address": null,
    "status": "pending",
    "updated_at": 1778102233,
    "type": "delivery",
    "costumer_id": "069fb788-0940-7431-8000-c1d2a353b3f0",
    "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
    "notes": "",
    "price": "38.94",
    "table_number": null,
    "created_at": 1778102233
  }
]
```

4.7.4 GET /api/v1/orders/:order_id

Finalidade: Obter detalhes de um pedido específico

Permissão: JWT | Perfil: Manager, Waiter, Delivery

Status Codes: 200, 401, 403, 404

200 OK

```
{
  "channel": "app",
  "id": "069fbafd-9f9d-77fd-8000-f9429e2381b0",
  "items": [
    {
      "name": "Hambúrguer com Batata Frita",
      "quantity": 1
    },
    {
      "name": "Brigadeiro (6un)",
      "quantity": 1
    }
  ],
  "payment_method": "pix",
  "pointsEarned": 38,
  "delivery_address": null,
  "status": "pending",
  "updated_at": 1778102233,
  "type": "delivery",
  "costumer_id": "069fb788-0940-7431-8000-c1d2a353b3f0",
  "store_id": "069f3640-e1bc-7fa2-8000-a6059a7ab69a",
  "notes": "",
  "price": "38.94",
  "table_number": null,
  "created_at": 1778102233
}
```

4.8 Respostas de Erro Padrão

Os endpoints da API seguem um formato consistente para respostas de erro, utilizando códigos de status HTTP apropriados e mensagens de erro claras. As respostas de erro padrão incluem:

1. **401 Unauthorized:** Retornado quando o token de autenticação é inválido ou ausente, ou quando as credenciais fornecidas para login são incorretas.

```
{ "detail": "Email or password is invalid" }
```

```
{ "detail": "Invalid token" }
```

2. **403 Forbidden**: Retornado quando o usuário autenticado não possui permissão para acessar o recurso solicitado.

```
{ "detail": "Acesso Negado" }
```

3. **404 Not Found**: Retornado quando o recurso solicitado não existe.

```
{ "detail": "Resource not found" }
```

4. **422 Unprocessable Entity**: Retornado quando os dados enviados na requisição não atendem aos requisitos de validação definidos pelo servidor.

```
{  
  "detail": [  
    {  
      "type": "missing",  
      "loc": ["body", "field_name"],  
      "msg": "Field required"  
    }  
  ]  
}
```


5 CONCLUSÃO

A solução entregue responde com consistência à pergunta proposta pelo Estudo de Caso: um sistema que compreende a jornada do cliente, modela os processos de negócio e toma decisões técnicas fundamentadas. Todavia, há uma limitação reconhecida: a ausência de testes automatizados, a validação de endpoints foi realizada manualmente via Insomnia.

Apesar disso, o projeto atendeu aos requisitos funcionais e não funcionais estabelecidos, demonstrando todo o trajeto comum de uma aplicação de Food Service, desde a gestão de produtos, catálogo, funcionários, clientes, pedidos, até a entrega.

O desenvolvimento do Estudo de Caso proporcionou uma experiência prática valiosa, permitindo a aplicação de conceitos teóricos em um cenário realista.

Referências

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2014.

FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2015.

GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 2009.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions**. Boston: Addison-Wesley, 2015.